OS ORANGE ASSIGNMENT

Name: Sreyaesh Peruri

SRN: PES1UG24AM189

SECTION: D

Screenshots

1A

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
```

2A

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ xxd .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227d0221bbe9db5e5c4b9aafc12 | head -20
00000000: 626c 6f02 2031 3600 4865 6c6c 6f2c 205   blob 16.Hello, P
00000010: 4555 2d56 4353 210a                      ES-VCS!.
```

3A

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  test_sequence.sh
  untracked:  Makefile
  untracked:  object.c
  untracked:  .gitignore
  untracked:  commit.h
  untracked:  index.h
  untracked:  pes.h
  untracked:  pes.c
  untracked:  tree.c
  untracked:  index.c
  untracked:  test_tree.c
  untracked:  tree.h
  untracked:  README.md
```

3B

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ cat .pes/index
304444 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776770123 6 file1.txt
304444 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776770128 6 file2.txt
```

4A

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ v
v: command not found
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ echo "Hello" > hello.txt
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes add hello.txt
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes commit -m "Initial commit"
Committed: a3a06aa8a7d1... Initial commit
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes log
commit a3a06aa8a7d123732d1ea718e11f55bb901fdfe35830751f79d80413d3c08cd4
Author: Sreyaesh Peruri <PES1UG24AM189>
Date:   1776772052

    Initial commit

sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ echo "World" >> hello.txt
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes add hello.txt
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes commit -m "Add world"
Committed: 65d54bb24426... Add world
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ echo "Goodbye" > bye.txt
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes add bye.txt
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes commit -m "Add farewell"
Committed: d3fd274187e8... Add farewell
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ ./pes log
commit d3fd274187e8b53a5482453712e4843a14501f00e1a3e217d9eb1726580a0949
Author: Sreyaesh Peruri <PES1UG24AM189>
Date:   1776772150

    Add farewellr auth!

commit 65d54bb244261060291de43b302dfdacf8fddc6eb43922bef53db0eae5f0c225
Author: Sreyaesh Peruri <PES1UG24AM189>
Date:   1776772136

    Add worldellr auth!

commit a3a06aa8a7d123732d1ea718e11f55bb901fdfe35830751f79d80413d3c08cd4
Author: Sreyaesh Peruri <PES1UG24AM189>
Date:   1776772052

    Initial committial commitEA
```

4B

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/65/d54bb244261060291de43b302dfdacf8fddc6eb43922bef53db0eae5f0c225
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/8c/d6c87f76a579f425e4c471d784817b1f4038ea775340d7aa9699577c372fac
.pes/objects/a3/a06aa8a7d123732d1ea718e11f55bb901fdfe35830751f79d80413d3c08cd4
.pes/objects/c2/b3b19dc96c683910f546098cbd45b3e601530e7500e8cb8a8d0544b9daa3c4
.pes/objects/c4/77950a73c7da17c2adcb5ddb924fe37fcd3724a9e7b3cd9c51204c6e975877
.pes/objects/d3/fd274187e8b53a5482453712e4843a14501f00e1a3e217d9eb1726580a0949
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

4C

```
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ cat .pes/refs/heads/main
d3fd274187e8b53a5482453712e4843a14501f00e1a3e217d9eb1726580a0949
sreyaesh07@DESKTOP-5I39JFV:~/PES1UG24AM189-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Q5.1: Implementing pes checkout <branch>

A branch in PES is stored as a file in .pes/refs/heads/<branch> containing a commit hash.

Steps to implement checkout:

1. Check if the branch exists by verifying the file .pes/refs/heads/<branch>.
2. Update .pes/HEAD to point to the branch ref:
   ref: refs/heads/<branch>
3. Read the commit hash from the branch file.
4. Load the corresponding commit object and extract its tree hash.
5. Traverse the tree object and update the working directory:
   - Create directories and files as per the tree.
   - Overwrite existing files if needed.
6. Update the index (.pes/index) to match the checked-out tree.

Why this operation is complex:

- It must safely overwrite files in the working directory.
- It must detect and handle uncommitted changes.
- It requires recursive traversal of tree objects.
- It must maintain consistency between the working directory, index, and object store.

Q5.2: Detecting a Dirty Working Directory

To detect conflicts before checkout:

1. For each file in the index:
   - Compare its metadata (mtime and size) with the working directory.

    - If they differ, the file is modified.
2. Load the target branch's tree and compare:
    - Current index hash vs target tree hash.
3. If a file:
    - Modified in the working directory, and
    - Different in the target branch,

then a conflict exists.

Condition:

if (working file != index) AND (index != target tree)
→ conflict → abort checkout

This ensures no uncommitted work is lost.

Q5.3: Detached HEAD

Detached HEAD means HEAD contains a commit hash instead of a branch reference.

What happens when committing:

- New commits are created normally.
- However, no branch points to these commits.
- These commits become unreachable from any branch.

How to recover:

- Create a new branch pointing to the commit:

pes branch <new-branch-name> <commit-hash>

- Or manually update .pes/refs/heads/<branch> with the commit hash.

If not recovered, these commits may be deleted during garbage collection.

Q6.1: Garbage Collection Algorithm

Goal:

Remove unreachable objects from the object store.

Algorithm:

1. Initialize a set (e.g., hash set) called “reachable”.
2. Add all commit hashes from .pes/refs/heads/* to a queue.
3. Perform traversal:
    - For each commit:
        - Add commit hash to reachable set.
        - Add its tree hash.
        - Add parent commit (if exists).
    - For each tree:
        - Add all child object hashes (blobs and subtrees).
4. Continue until all reachable objects are visited.
5. Scan .pes/objects:
    - Delete any object not in the reachable set.

Data structure:

- Use a hash set for O(1) lookup of reachable objects.

Estimation:

For 100,000 commits and 50 branches:

- Traversal visits all commits.
- Each commit references at least one tree.
- Each tree references multiple blobs.
- Total objects visited can be several hundred thousand.

Q6.2: GC Race Condition

Running garbage collection during a commit is dangerous due to race conditions.

Example race condition:

1. Commit process:
    - Writes new blob and tree objects.

    - Has not yet updated HEAD or branch reference.
2. GC runs concurrently:
    - Scans references and does not see new objects as reachable.
    - Deletes these newly created objects.
3. Commit then updates HEAD:
    - Now HEAD points to non-existent objects.

Result:

- Repository becomes corrupted.

How Git avoids this:

- Uses locking mechanisms during critical operations.
- Marks new objects as “in-progress” or protects them temporarily.
- Runs garbage collection only when no write operations are active.
- Uses atomic object creation and reference updates to avoid inconsistency.